Analisi e Correzione del Codice Python

Documentazione testuale di revisione logica, strutturale e sintattica

1. Analisi Iniziale delle Righe e Importazioni

Riga 1: Riguardo all'importazione del modulo, tutto risulta corretto in quanto viene importato il modulo necessario per gestire date e ore all'interno dello script.

Riga 3: Viene evidenziato un errore bloccante in quanto mancano i due punti alla fine della riga. Trattandosi di un'istruzione condizionale o iterativa, Python richiede tassativamente la presenza dei due punti, generando altrimenti un errore di sintassi immediato che impedisce l'avvio dello script.

> **Nota sull'acquisizione dell'input (Righe 3-4):** Durante la ricezione dei dati inseriti dall'utente, si nota che la stringa ricevuta non viene pulita da spazi superflui. L'assenza di metodi di normalizzazione rende il confronto vulnerabile a spazi accidentali digitati dall'utente.

2. Flusso di Esecuzione e Gestione dello Scope

Righe 5-7: Questa sezione gestisce la condizione di uscita dal ciclo iterativo qualora l'utente digiti la parola chiave prevista per terminare il programma.

Righe 8-9: Si rileva un errore critico di esecuzione poiché la funzione dedicata all'assistente viene chiamata prima di essere effettivamente dichiarata nel flusso del codice. Poiché l'interprete legge dall'alto verso il basso, questo ordine genera un errore di riferimento non trovato.

3. Analisi della Funzione e delle Specifiche Interne

Esaminando il blocco interno della funzione e le righe successive, emergono diversi aspetti da correggere:

- **Dichiarazione della funzione e riorganizzazione:** Tutto ciò che è associato alla definizione della funzione, ovvero dalla riga 10 fino alla fine dello script, andrebbe completamente spostato in alto, precisamente alla riga 2 (subito dopo l'importazione iniziale), posizionando l'intero blocco prima del ciclo iterativo principale.
- **Gestione della data:** Nel richiamare il modulo della data, l'utilizzo originario presentava un metodo non esistente nella libreria standard ufficiale. È necessario ricorrere alla corretta sottomartellata di libreria per evitare errori di attributo.
- **Gestione dell'orario:** La richiesta dell'orario corrente sfrutta le funzioni native del sistema di data e ora, sebbene presenti una struttura che può essere resa più lineare.
- **Pulizia del testo:** È stata completamente rimossa la stringa contenente un carattere anomalo o invisibile che poteva compromettere la stabilità delle risposte testuali.
- **Struttura logica e allineamento:** La gestione dei condizionali ramificati restituisce i valori attesi. Inoltre, la scritta del comando `break` va spostata più avanti verso destra, posizionandola esattamente sotto la riga soprastante per rispettare la corretta rientro sintattico.

4. Conclusioni e Linee Guida per il Refactoring

La revisione complessiva del codice richiede dunque di ristrutturare l'ordine delle componenti: spostare tutte le definizioni funzionali dalla riga 10 in poi direttamente alla riga 2 prima di avviare i cicli di interazione, correggere i richiami alle librerie di sistema, sanificare l'input utente tramite i metodi appositi delle stringhe, e assicurarsi che l'istruzione di `break` sia spostata più avanti per allinearsi perfettamente al blocco di codice soprastante.

Appunti Python: Correzione Errori e Gioco dei Numeri

Ecco un promemoria degli errori che facevo più spesso quando ho iniziato a scrivere script in Python e come sistemarli, più uno script completo che ho scritto per far indovinare un numero segreto al PC.

1. Variabili non definite e maiuscole

All'inizio sbagliavo spesso a scrivere le variabili. Se definisco una variabile minuscola e poi la richiamo con la maiuscola, Python si blocca e dà errore perché è case-sensitive.

```
# Errore tipico:
nome_utente = "Marco"
print(Nome_utente)  # Sbagliato! La N è maiuscola

# Come si corregge:
nome_utente = "Marco"
print(nome_utente)  # Corretto
```

2. Virgolette e stringhe aperte

Bisogna sempre stare attenti a chiudere le virgolette quando si stampa un testo, altrimenti dà errore di sintassi subito prima di far partire lo script.

```
# Errore:
print("Ciao a tutti!)  # Mancano le virgolette di chiusura

# Corretto:
print("Ciao a tutti!")
```

3. Conversione dei tipi (str e int)

Quando uso `input()`, Python salva sempre il valore come testo (stringa). Se voglio farci dei calcoli o confrontarlo con dei numeri, devo convertirlo usando `int()`.

```
# Errore:
eta = input("Quanti anni hai? ")
prossimo_anno = eta + 1  # Errore! Non si può sommare testo e numero

# Corretto:
eta = int(input("Quanti anni hai? "))
prossimo_anno = eta + 1
```

4. Indentazione nei blocchi if e while

In Python gli spazi all'inizio della riga contano. Dopo i due punti : bisogna sempre rientrare di quattro spazi, altrimenti il codice non capisce cosa sta dentro al blocco.

5. Codice Completo: Gioco Indovina il Numero

Questo è lo script che ho scritto per esercitarmi con i cicli `while`, la libreria `random` e la gestione dei tentativi dell'utente. Genera un numero casuale tra 1 e 100 e dà degli indizi ogni volta che si sbaglia.

```
import random

# Genero il numero segreto tra 1 e 100
numero_segreto = random.randint(1, 100)
tentativi_fatti = 0
indovinato = False

print("=== GIOCO: INDOVINA IL NUMERO ===")
print("Ho pensato a un numero tra 1 e 100. Prova a indovinarlo!")

# Ciclo finché l'utente non indovina
while not indovinato:
    scelta = input("Inserisci il tuo tentativo: ")

    # Verifico che l'utente abbia inserito un numero
    if not scelta.isdigit():
        print("Per favore inserisci un numero valido!")
        continue

    numero_utente = int(scelta)
    tentativi_fatti += 1

    if numero_utente < numero_segreto:
        print("Troppo basso! Prova un numero più alto.")
    elif numero_utente > numero_segreto:
        print("Troppo alto! Prova un numero più basso.")
    else:
        indovinato = True
        print(f"Bravo! Hai indovinato il numero {numero_segreto} in {tentativi_fatti}
tentativi!")
```

Cosa fa lo script:

- Usa `random.randint(1, 100)` per scegliere il numero.
- Controlla con `isdigit()` per evitare che il programma si pianti se scrivo lettere al posto dei numeri.
- Usa un ciclo `while` che continua finché la risposta non è corretta.